Trabajo Práctico — Parte 3: Árbol de clock y routing con LibreLane (sky130)

En la parte 1 sintetizaste el CPU Ibex, y en la parte 2 hiciste su floorplan y ubicaste todas sus celdas. En esta parte terminás la implementación física: construís el **árbol de clock** (CTS), **ruteás** todos los cables y hacés el análisis de timing final con los cables reales.

Se trabaja **solo con el Ibex y la biblioteca `hd`**, partiendo del placement del TP2: acá no se sintetiza ni se ubica nada de nuevo.

Objetivos

Al terminar este trabajo práctico deberías poder:

1. Explicar para qué sirve el **árbol de clock** y describir el que construye la herramienta: buffers, niveles, latencia y skew.
2. Explicar la diferencia entre el timing con clock **ideal** y con clock **propagado**, y su efecto en setup y hold.
3. Explicar las dos etapas del **routing** (global y detallado) e interpretar sus resultados: uso de cada capa, overflow, largo de cables, vías y violaciones de DRC.
4. Explicar qué es el **efecto antena** y cómo se repara.
5. Comparar el **timing a lo largo de todo el flujo**, desde la síntesis hasta el layout ruteado, y en las tres esquinas.
6. Usar la **GUI de OpenROAD** para ver el árbol de clock y los cables ruteados.

Antes de empezar

- El repositorio es https://github.com/miguel9554/up-tp3. Se usan el mismo contenedor y el mismo PDK del TP1 y del TP2: no hay que instalar nada nuevo.
- Leé **todo el `README.md`**. Ahí está cómo importar el placement del TP2, qué hace cada paso del flujo, qué contiene cada reporte y qué hay de nuevo en la GUI. Este TP supone que ya lo leíste.
- Necesitás la corrida de placement de referencia del TP2 (`pl_ref_...`, ejercicio 2.1 del TP2), **con la configuración original**. Si la borraste, volvé a correr `make place NAME=ref` en el proyecto del TP2.
- Todos los comandos se corren desde la raíz del proyecto. La GUI tiene que abrirse desde una sesión gráfica (sección 5 del `README.md`).

Cosas a tener en cuenta durante todo el TP

- **`make route` tarda** (de 25 minutos a más de una hora, según tu computadora). Corrélo una sola vez, con `NAME=ref`, y dejalo terminar. Mientras tanto podés hacer el ejercicio 1 con la corrida de `make cts`.

- No hace falta modificar ningún archivo del proyecto. Si lo hiciste, volvé a la versión original antes de correr (`git diff` no tiene que mostrar nada).
- Los STA intermedios (después de CTS y del routing global) analizan solo la esquina `tt`. El STA final, después del routing detallado, analiza las tres. Para comparar etapas, usá siempre `tt`.
- Las imágenes de la GUI se guardan con `save_image <archivo>.png` en su consola Tcl.

Entrega

Un **informe en PDF** que contenga:

- Las **tablas** pedidas, con los valores obtenidos y el nombre de la corrida de la que salen.
- Las **respuestas** a cada pregunta, justificadas con datos de los reportes o de la GUI (citá el archivo y el valor). Una respuesta sin justificación con datos no se considera completa.
- Las **capturas de la GUI** pedidas, con una breve descripción de qué se ve en cada una.

Las preguntas están numeradas (**P1**, **P2**, ...): usá esa numeración en el informe.

Uso de herramientas de IA y defensa del trabajo

- **Está permitido usar chats o agentes de IA** (ChatGPT, Claude, Copilot, etc.) para hacer este trabajo: para entender conceptos, interpretar reportes, escribir comandos o revisar el informe.
- En la **evaluación final el trabajo deberá ser defendido** oralmente. Vas a tener que explicar cada respuesta y cada número, de dónde sale y qué significa, sin ayuda. Usá la IA para aprender, no para entregar algo que no entendés.

Ejercicio 0: Importar el placement del TP2

Qué hacer:

1. Importá la corrida de placement de referencia del TP2:

   ```
   make import-tp2 TP2_RUN=../up-tp2/designs/ibex/runs/<corrida pl_ref_...>
   ```

2. Compará la configuración y el SDC con los del TP2:

   ```
   diff ../up-tp2/designs/ibex/constraints.sdc designs/ibex/constraints.sdc
   diff ../up-tp2/designs/ibex/config.json designs/ibex/config.json
   ```

Qué se debe lograr: saber de qué parte esta etapa y qué cambia en las restricciones.

- **P1.** ¿Qué corrida del TP2 importaste? ¿De qué corrida del TP1 sale su netlist (`designs/ibex/tp2/SOURCE`)?
  - ¿Qué cambió en el SDC? En la P3 del TP2 explicaste por qué antes de CTS convenía el clock **ideal**. ¿Por qué ahora hay que usar el **propagado**? ¿Qué pasaría con el análisis de timing si dejáramos el clock ideal después de construir el árbol?
  - ¿Qué variables se sacaron de `config.json` respecto del TP2? ¿Por qué ya no hacen falta? ¿Dónde está guardada ahora esa información (die, pines, macros, PDN)?

Ejercicio 1: Árbol de clock (CTS)

1.1 Correr CTS

Qué hacer:

```
make cts NAME=ref
```

Tarda 2-3 minutos. Leé `cts.md` y `timing.md` de la corrida y abrila en la GUI (`make gui`).

Qué se debe lograr: entender qué construye CTS y por qué.

- **P2.** Los nets de clock. En la sección *Clock nets found* de `cts.md` aparecen tres nets.
  - ¿Qué maneja cada uno? Buscá en el netlist del TP2 (`designs/ibex/tp2/ibex_top.nl.v`) a qué pines se conecta `clk_i` y cuál es la celda que genera el net `clk` (pista: la viste en la P3 del TP2).
  - ¿Por qué hay un clock "con gating" (`clk`) y otro sin gating (`clk_i`)? ¿Qué se conecta a cada uno?
  - Las SRAM tienen dos pines de clock (`clk0` y `clk1`). ¿Se usan los dos? (Recordá la P5 del TP2.)
- **P3.** El árbol. Completá esta tabla con los datos de `cts.md`:

Ítem	Valor
Buffers de clock insertados	
Celdas usadas (tipo y cantidad)	
Profundidad del árbol de `clk` (buffers entre la raíz y un flip-flop)	
Buffers hoja (*leaf*) de `clk`	
Flip-flops por buffer hoja, en promedio	
Buffers de retardo (*delay buffers*)	

  - ¿Por qué CTS usa celdas `clkbuf` y no los buffers comunes (`buf`)? Buscá en la documentación de la biblioteca o pensá en qué le importa a un buffer de clock: ¿retardo? ¿igualdad entre flanco de subida y de bajada?
  - ¿Para qué sirven los *delay buffers* y las *dummy loads*? (No agregan lógica: pensá en el objetivo de un árbol de clock.)
  - Buscá en la GUI el net `clk` (botón **Find**) y mirá su fanout en el Inspector. En la P13 del TP2 maneja 2352 pines. ¿Cuántos tiene ahora? ¿A qué se conecta?

1.2 Latencia y skew

- **P4.** En `cts.md`, sección *Clock latency and skew after CTS*:
  - ¿Cuál es la latencia mínima y la máxima del flanco de subida? ¿A qué pines llegan? ¿Cuánto es el skew?
  - Explicá con tus palabras qué es la **latencia** y qué es el **skew**. ¿Por qué a CTS le importa mucho más que el skew sea chico que la latencia?
  - En la P3 del TP2 estimaste qué pasaba con un solo clock gate manejando 2351 flip-flops (el STA daba ~42 ns de latencia). Compará con la latencia que hay ahora. ¿Qué logró el árbol?
- **P5.** El árbol en la GUI. Abrí la corrida (`make gui`).
  - En Display Control → `Nets`, dejá marcado solo `Clock` y ocultá las celdas (`Instances`). Incluí una captura del árbol completo. ¿Se ve la forma de "H" que usa el algoritmo (*H-tree*)? ¿Dónde hay más buffers? Comparalo con el mapa de densidad que viste en la P11 del TP2.
  - Abrí **Windows → Clock Tree Viewer** y apretá `Update`. Incluí una captura. ¿Qué representa el eje vertical? ¿Se ven los distintos niveles del árbol? ¿Los flip-flops (las hojas) quedan todos a la misma altura? ¿Por qué no?

1.3 Timing después de CTS

- **P6.** Completá con `timing.md` de la corrida de CTS:

Esquina `tt`	TP2 (placement)	Después de CTS
Setup WS (ns)		
Hold WS (ns)		
Latencia de clock mín / máx (ns)		
Skew (ns)		

  - El slack de setup **empeoró**. Compará el camino crítico de setup del TP2 (`designs/ibex/tp2/reports/timing.md`) con el de ahora (`timing.md`): ¿empiezan y terminan en el mismo lugar? ¿En qué instante llega el clock a la SRAM (`clk0`) en cada uno? ¿Cambió el retardo del resto del camino?

- El camino termina en un pin de salida (`instr_addr_o`). El *data required time* ¿cambió? ¿Por qué la latencia del clock perjudica justamente a los caminos que van de un registro a una salida?
- En un camino entre dos flip-flops, ¿la latencia también empeoraría el slack? ¿Qué es lo que importa en ese caso?
- Según la sección *Timing repair after CTS* de `cts.md`, ¿hizo falta reparar setup o hold? ¿Por qué el hold sigue cumpliéndose aunque ahora haya skew? (Compará el skew con el slack de hold.)

Ejercicio 2: Routing

2.1 Correr el routing

Qué hacer:

```
make route NAME=ref
```

Tarda bastante (sección 3.3 del `README.md`): dejalo corriendo. Cuando termine, leé `qor.md`, `routing.md` y `timing.md` de la corrida y abrila en la GUI.

Qué se debe lograr: entender las dos etapas del routing y saber leer sus resultados.

- **P7.** Routing global (`routing.md`, sección *Global routing*).
  - Completá:

Capa	Dirección	Recursos (después del ajuste)	Uso (%)	Overflow
met1				
met2				
met3				
met4				
met5				

  - ¿Por qué cada capa tiene una sola dirección, y por qué se alternan? (Relacionalo con lo que viste de los pines en la P2 del TP2.)
  - ¿Qué capas se usan más y cuáles menos? ¿Por qué met5 tiene tan pocos recursos y casi no se usa? (Pensá en qué viste en met5 en la P9 del TP2, y en el *track pitch* de cada capa, que está en el log del paso 05: `GRT-0088`.)
  - ¿Qué significa que el overflow sea 0? ¿Qué es el "ajuste" (*global adjustment*) del 30 % y por qué conviene dejar ese margen?

- **P8.** Largo de cables. Completá con `qor.md` y `routing.md`:

Etapa	Wirelength (µm)
HPWL después del placement (TP2)	
HPWL después de CTS	
Routing global	
Routing detallado	

  - ¿Por qué el HPWL aumentó después de CTS?
  - El HPWL es una estimación optimista: ¿por qué el cable real de un net con muchos pines es más largo que el semiperímetro de su rectángulo?
  - El routing global da un largo mayor que el detallado. ¿Por qué puede pasar eso? (Pensá en con qué resolución trabaja el routing global: las gcells.)

- **P9.** Routing detallado (`routing.md`, sección *Detailed routing*).
  - ¿Cuántas violaciones de DRC había después de la primera iteración, y cuántas al final? ¿De qué tipo eran la mayoría y en qué capa? Explicá qué es un *short* y qué es una violación de *metal spacing*.
  - ¿Por qué el router hace varias rutas completas ("Route" 0 a 3) y no una sola? (Mirá la columna *Diodes added before* y la sección 2.2.)
  - Completá la tabla de largo y vías por capa. ¿En qué capas va la mayor parte del cable? ¿Por qué casi todas las vías son de `li1` a `met1` y de `met1` a `met2`? (Pensá en en qué capa están los pines de las celdas estándar.)
  - ¿Cuánto tardó el routing detallado en tu computadora (`10-openroad-detailedrouting/runtime.tx`
    ¿Y todo el `make route`? ¿Qué etapa es la más cara?

- **P10.** El layout ruteado en la GUI (`make gui RUN=rt_ref_... STEP=10`).
  - Incluí una captura del chip completo con todas las capas y otra con **una sola capa** (por ejemplo `met2`). ¿Se cumple la dirección preferida que anotaste en la P7?
  - Hacé zoom sobre unas pocas celdas estándar hasta ver sus pines y las vías (captura). ¿En qué capa están los pines de las celdas? ¿Cómo sube un cable desde un pin hasta met2?
  - ¿Pasan cables por encima de las SRAM? ¿De qué nets y en qué capa? (Recordá hasta qué capa usa la SRAM, P9 del TP2.) ¿Qué consecuencia tiene para los cables que tienen que cruzar de un lado al otro de una fila de macros?
  - Buscá `clknet_0_clk` (botón **Find**). En el DEF del paso 10 (`10-openroad-detailedrouting/ibex_top.def`) aparece con `+ USE CLOCK + NONDEFAULTRULE CTS_NDR_2`: los nets de clock se rutean con una regla especial, con más separación a los vecinos. ¿Para qué crees que sirve?

2.2 Antenas, relleno y chequeos

- **P11.** El efecto antena (definición en la sección 4 del `README.md`).
  - Explicá con tus palabras por qué un cable largo puede dañar la compuerta de un transistor **durante la fabricación**, y por qué el problema depende de la capa de metal (se fabrican de abajo hacia arriba).
  - Completá con `routing.md`:

Ítem	Valor
Nets con violaciones de antena después del routing global	
Jumpers insertados (suma de los pasos)	
Diodos insertados después del routing global	
Diodos insertados durante el routing detallado	
Nets con violaciones al final	

  - ¿Qué hace un *jumper* y qué hace un diodo? ¿Por qué el diodo descarga el cable?
  - Mirá los nets de `06-openroad-checkantennas/reports/antenna_summary.rpt`. ¿Qué tipo de nets tienen las peores violaciones? ¿A qué celdas van? (Relacionalo con la P15 del TP2.)
  - Buscá en la GUI alguna celda de diodo (sus instancias se llaman `ANTENNA_*`). ¿Dónde está, respecto del pin que protege?
- **P12.** Relleno y chequeos (`routing.md`, sección *Checks after detailed routing*).
  - ¿Cuántas celdas de relleno se insertaron (paso 18)? ¿De qué tipos son (`fill`, `decap`)? ¿Para qué sirve cada tipo? ¿Por qué no se pueden dejar huecos vacíos en las filas? (Pensá en los pozos de la P8 del TP2.) Comparalo en la GUI con `STEP=10` y `STEP=18`.
  - Hay pines desconectados (*disconnected pins*), pero ninguno es crítico. Mirá `14-odb-reportdisconnectedpins/full_disconnected_pins_table.txt`: ¿de qué instancias son? ¿Por qué es normal que estén desconectados?

Ejercicio 3: Timing final

Qué se debe lograr: seguir el timing del diseño a lo largo de todo el flujo, y entender qué aporta cada etapa.

- **P13.** Completá esta tabla con la sección *Timing at each stage* de `timing.md` de la corrida de routing (esquina `tt`):

Etapa	Setup WS (ns)	Hold WS (ns)	Skew (ns)	Max slew vio	Max cap vio
TP1 (síntesis)					
TP2 (placement)					
Después de CTS					
Después del routing global					
Después del routing detallado + RCX					

  - Explicá **cada** cambio del setup de una fila a la siguiente. Para las dos primeras filas podés usar tu P14 del TP2.
  - El camino crítico ¿sigue siendo el mismo que en el TP2? Compará en los tres reportes (TP2, después de CTS y final) el instante en que llega el clock a la SRAM (`clk0`), y el retardo y el slew del net largo que va al buffer de salida (`net405`, entrada de `output405`). ¿Qué parte del empeoramiento final se debe al clock y qué parte a los cables?
  - ¿Por qué el skew creció después del routing?
  - ¿Qué es lo que hace el STA final más "real" que los anteriores? ¿Qué tiene el archivo SPEF (`20-openroad-rcx/nom/ibex_top.nom.spef`)? Para achicarlo, el SPEF reemplaza cada nombre por un número: buscá la línea que termina en `net405` (en la sección `*NAME_MAP`), anotá su número (`*<n>`) y buscá `*D_NET *<n>`. ¿Qué datos tiene? ¿Cuál es la capacidad total del net? ¿A qué pines se conecta? ¿Por qué tiene tantos diodos?

- **P14.** Violaciones nuevas.
  - Después del routing detallado aparecen muchas más violaciones de

max slew y max cap que antes. ¿En qué pines están? (Buscalas en `21-openroad-stapostpnr/nom_tt_025C_1v80/checks.rpt`.) ¿Qué tienen en común esos nets? Compará con los nets más largos de `routing.md`.
  - ¿Por qué no aparecieron antes, con los cables estimados? ¿Cómo influyen los diodos de antena (fijate si aparecen pines `ANTENNA_*/DIODE`)?
  - ¿Por qué nadie las reparó? Mirá las variables `RUN_POST_GRT_DESIGN_REPAIR` y `RUN_POST_GRT_RESIZER_TIMING` en la sección 3.1 del `README.md`. ¿Qué harías en un diseño real antes de fabricarlo?

- **P15.** Esquinas. Completá con la sección *Corners, after detailed routing* de `timing.md`:

Esquina	Setup WS (ns)	Hold WS (ns)	Max slew vio	Max cap vio
`ss`				
`tt`				
`ff`				

  - ¿Qué esquina es la peor para setup y cuál para hold? ¿Por qué? (Pensá en qué pasa con el retardo de las celdas en cada una.)
  - ¿Por qué en `ss` hay tantas más violaciones de max slew?
  - Con 100 ns de período, ¿el diseño cumple el timing en las tres esquinas? ¿Qué número mirarías para decidir si podría andar a una frecuencia mayor?

Criterios de evaluación

Se valora especialmente que las respuestas se apoyen en **números de los reportes y observaciones en la GUI**, y que relacionen lo que pasa en cada etapa con los conceptos de la materia: latencia y skew, setup/hold, largo de cables, resistencia y capacidad, congestión y reglas de diseño.